Map-Reflect: Schema Mapping using Induced Reflection

Balaji Ganesan*
IBM Research
India
bganesa1@in.ibm.com

Avirup Saha*
IBM Research
India
avirup.saha2@ibm.com

Gunjan Siddharth
Vellore Institute of Technology
India
gunjan.siddharth2022@vitstudent.ac.in

Ravindran Jagannathan
IBM Software
USA
ravi.jagannathan@ibm.com

Amit Paradkar
IBM Research
USA
paradkar@us.ibm.com

Pratibha Moogi
IBM Research
India
pratibha.moogi@ibm.com

ABSTRACT

We study many-to-many schema mapping, a setting where multiple source types/fields may correspond to overlapping targets and vice versa—cases that retrievers and single-chain API sequencers fail to disambiguate. Prior approaches use self-reflection where models critique their own output, but self-reflection lacks external grounding and models may reinforce their own errors. We introduce Map-Reflect, an *induced reflection* framework that performs schema mapping through a two-phase approach: an initial greedy mapping followed by reflection induced by three external signals—neighbourhood graph validation, error analysis (LLM-as-judge, schema deployment, query execution), and uncertainty quantification. Our approach is schema-agnostic, applying uniformly to REST APIs, databases, and other structured schemas. Map-Reflect applies domain-aware clustering to bound candidate sets and runs multi-stage induced reflection to achieve comprehensive many-to-many coverage without spurious mappings. We evaluate on multiple datasets including new enterprise benchmarks for API and database schema mapping. Induced reflection consistently lifts accuracy over non-reflective baselines while remaining model-agnostic. The framework emits interpretable rationales through a wiki validated by business stakeholders for human-in-the-loop validation in enterprise settings.

*Equal contribution.

PVLDB Reference Format:
Balaji Ganesan*, Avirup Saha*, Gunjan Siddharth, Ravindran Jagannathan, Amit Paradkar, and Pratibha Moogi. Map-Reflect: Schema Mapping using Induced Reflection. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

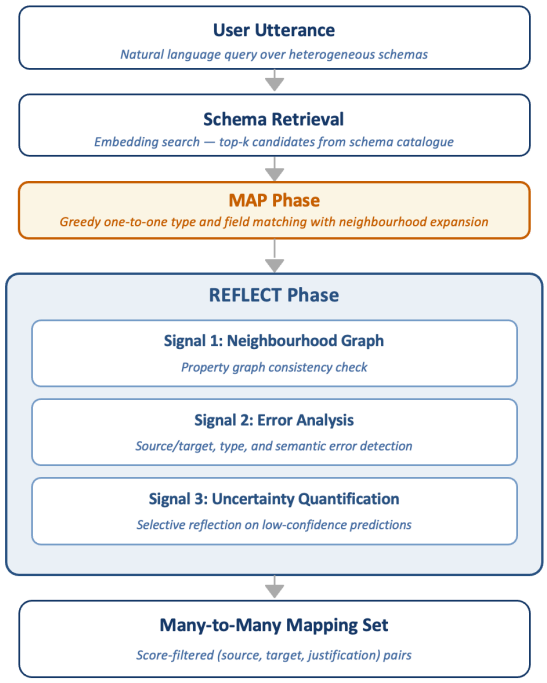


Figure 1: Map-Reflect pipeline: greedy MAP followed by signal-driven REFLECT.

1 INTRODUCTION

Organizations rely on diverse API ecosystems and databases that must be integrated for effective data access. Schema mapping—identifying equivalent entity types across different schemas—is critical for building unified data access layers where users query heterogeneous sources without knowledge of underlying schema differences. This capability is increasingly important for AI agents and tool-calling scenarios, where LLMs must select and invoke APIs across multiple services.

Traditional approaches focus on database integration with column-level mapping, but enterprise environments require schema retrieval and multi-faceted mapping across both APIs and databases. For example, Site Reliability Engineers managing hybrid cloud environments must query across multiple observability platforms, each with distinct schemas. The complexity of mapping across these disparate systems creates significant overhead.

The Many-to-Many Challenge. Prior work on schema mapping focuses on one-to-one alignments or single-chain API sequencing. Real enterprise environments require *many-to-many mappings* where multiple source types/fields correspond to overlapping target structures. For example, `Incident` from one system

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

may correctly map to both `Alert` and `Event` in another. Greedy one-to-one mapping leaves too many elements unmapped, while naive approaches create spurious mappings that break query semantics.

Recent advances in tool calling with large language models show promise for API-related tasks. ToolLLM [27] introduced the ToolBench dataset with a BERT-based neural API retriever while AnyTool [5] leverages GPT-4 [24]'s function calling with hierarchical API organization and self-reflection. However, these methods focus on sequencing a single chain of calls; they do not address systematic alignment of schema elements across products. Critically, self-reflection mechanisms lack external grounding—the model critiques its own output without validation against external signals, potentially reinforcing errors. Further, these approaches require expensive models like GPT-4, limiting practical applicability.

Building a unified Data Access Layer requires mapping schemas across both APIs and databases. We address this challenge through experiments on API datasets (ITOpsGraph, NL2GQL, ToolBench) and database schemas (OMOP). Our ITOpsGraph benchmark provides schema mapping across heterogeneous enterprise APIs, featuring (i) deeply nested type hierarchies, (ii) domain-specific terminology where semantically equivalent concepts have different names, and (iii) many-to-many mappings where a single source type correctly maps to multiple targets. Our wiki-based SME annotation provides continuous ground truth refinement.

We introduce Map-Reflect, an *induced reflection* framework that addresses these limitations through a two-phase approach (Figure 1). In the first *Map* phase, we greedily find the best mapping for each source type and its fields. In the second *Reflect* phase, we induce reflection using three external signals to achieve comprehensive many-to-many mapping: (1) a *neighbourhood graph* that validates mapping assumptions, (2) *error analysis* from LLM-as-judge evaluation, schema deployment, and query execution, and (3) *uncertainty quantification* to identify contested mappings. Unlike self-reflection, induced reflection is externally grounded, validating mappings against objective signals.

Our contributions are: (1) Map-Reflect, an induced reflection framework for many-to-many schema mapping using three external signals (neighbourhood graph, error analysis, uncertainty quantification), (2) a two-phase architecture (greedy Map followed by induced Reflect) that addresses the coverage vs. precision tradeoff, (3) a wiki-based human-in-the-loop system for continuous ground truth refinement with full provenance tracking, (4) new enterprise benchmarks for API and database schema mapping (ITOpsGraph, NL2GQL, ToolBench, OMOP), and (5) comprehensive evaluation demonstrating that induced reflection outperforms self-reflection and non-reflective baselines.

2 RELATED WORK

ToolLLM [27] introduced the ToolBench dataset with an API Retriever for natural language instructions. AnyTool [5] incorporates self-reflection for reliability, but lacks external grounding—the model critiques its own output without validation against type-system checks or uncertainty measures, potentially reinforcing errors. These approaches focus on API sequencing, not schema mapping.

Classical Schema Matching. Cupid [20], COMA [4], and iMAP [3] established multi-signal matching combining linguistic, structural, and domain-specific similarity. However, these systems assume flat relational schemas and produce one-to-one correspondences, limiting applicability to nested API schemas with many-to-many relationships. Map-Reflect builds on multi-signal intuition but uses LLM-driven reasoning with externally grounded reflection.

DynaMap [22], embedding-based approaches [9], and SMUTF [37] use dynamic programming, word vectors, and XGBoost classifiers respectively, but struggle with contextual relationships.

API retrieval and sequencing have been addressed by embedding-based retrievers [15, 26], fine-tuned methods [27], and graph-based approaches [21]. Pre-LLM API mapping work includes document-based [18], knowledge graph [16], and GNN approaches [30].

LLM-Based Schema Mapping. LSM [36] uses fine-tuned BERT encoders; Magneto [17] combines SLMs with LLM reranking; ReMatch [31] and GraphRAGToolFusion [19] add retrieval mechanisms. MatchMaker [29] introduced compositional LLM programs with zero-shot self-improvement, but lacks external grounding signals. For many-to-many matching, LLMatch [34] uses Rollup/Drilldown abstraction, while SCHEMORA [7] combines BM25 and FAISS retrieval. Both target relational schemas and require upfront schema restructuring.

Positioning. Map-Reflect differs by beginning with greedy one-to-one mapping and inducing revision through three externally grounded signals. This requires no elaborate preprocessing—new schemas need only parsing and optional RAG enrichment. Map-Reflect is schema-agnostic across REST APIs, GraphQL, and databases, using induced reflection (neighbourhood validation, error analysis, uncertainty quantification) rather than self-reflection.

3 MAP-REFLECT FRAMEWORK

Map-Reflect employs a two-phase architecture designed to address the many-to-many mapping challenge. The first phase (Map) performs greedy one-to-one mapping, which is fast but leaves many elements unmapped and may create spurious mappings. The second phase (Reflect) induces reflection using three external signals to refine the initial mapping into comprehensive many-to-many coverage. Given API specifications (OpenAPI for REST, SDL for GraphQL), Map-Reflect performs domain and structural analysis to create clusters, then applies the two-phase mapping process.

Unlike GPT-4–centric tool agents, Map-Reflect is a service-oriented architecture whose components are invoked but never trained. A user utterance u is processed in five stages. First, a context extractor splits u into source and destination phrases by prompting a hosted chat model. Second, each phrase is embedded and queried against a Chroma vector index, yielding neighbourhoods $\mathcal{S}_{\text{src}}$ and $\mathcal{S}_{\text{dst}}$. Third, a reasoning call evaluates every pair in $\mathcal{S}_{\text{src}} \times \mathcal{S}_{\text{dst}}$, assigns a confidence γ_0, and produces justifications—this is the greedy Map phase. Fourth, the Reflect phase collects three external signals and induces reflection to refine mappings.

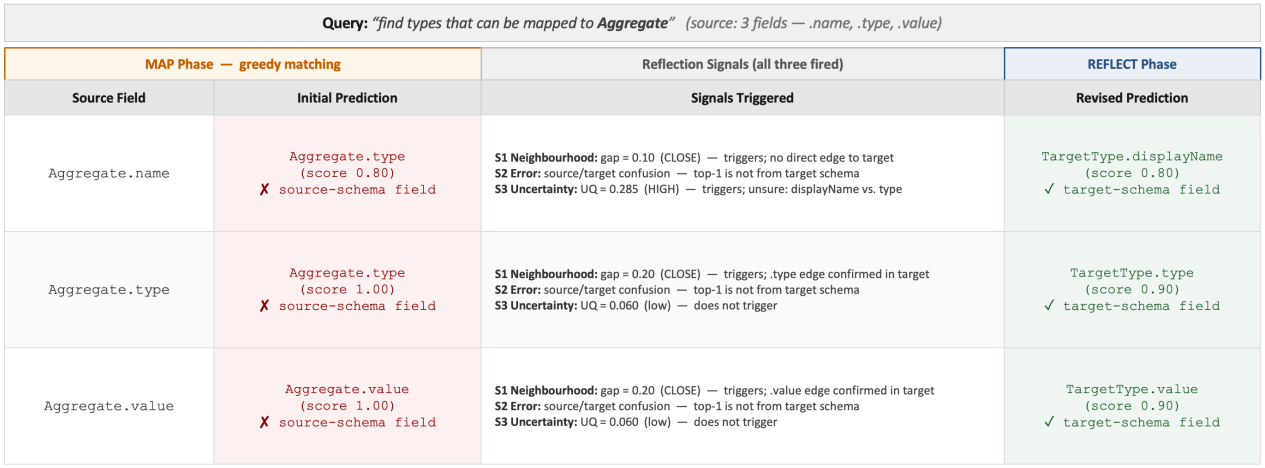


Figure 2: Induced reflection example on ITOpsGraph. The MAP phase produces incorrect initial predictions (mapping to source-schema fields). All three signals trigger: neighbourhood gap <0.2, source/target confusion error, and high uncertainty (0.285). The REFLECT phase corrects predictions to target-schema fields with improved confidence scores.

Finally, mappings with scores below threshold τ are pruned, yielding many-to-many coverage.

3.1 Induced Reflection Mechanism

The induced reflection mechanism is Map-Reflect's key innovation (Figure 2). Unlike self-reflection where models critique their own output without external validation, induced reflection uses three external signals to ground the refinement process. This ensures that mappings are validated against objective evidence rather than the model's own potentially erroneous judgments.

Reflection is induced by three external signals:

Signal 1: Neighbourhood Graph Validation. For each candidate mapping (s, t), we retrieve k-nearest neighbours from the graph store populated from schema mapping output. We validate assumptions by checking: Do neighbours of source type also map to neighbours of target type? Are there alternative targets in the neighbourhood with similar structure? Are there unmapped neighbours that should be included? If neighbourhood evidence contradicts the greedy mapping assumption, we add alternative mappings (achieving many-to-many) or remove inconsistent ones.

Signal 2: Error Analysis. We collect error signals from three sources at different stages of the pipeline:

(i) Initial Ranking Error Detection. Before reflection, we analyze the greedy mapping's initial ranking to detect specific error patterns that indicate the top prediction may be wrong:

- **Source/Target Product Mismatch**: If the top-ranked candidate belongs to the source product rather than the target product, the model matched by name similarity instead of finding the semantic equivalent in the target schema. This is a critical error—the model confused which direction to map.
- **Foreign Key Misalignment**: For fields ending in `_id` that reference another table (e.g., `concert_stadium_id`), we verify the prediction maps to the primary key of the referenced table (e.g., `stadium.id`), not to another foreign key in the source table.
- **Name Similarity Without Semantic Match**: When the top prediction has high lexical overlap with the source field but the model's reasoning mentions "name", "similar", or "match", we flag this as potentially superficial matching that ignores semantic meaning.
- **Type Mismatch Indicators**: If field names contain type hints (e.g., `_string`, `_int`, `_date`) and the source and target type hints disagree, we flag potential type incompatibility.
- **Low Confidence Warning**: When the top prediction's confidence score falls below 0.6, we flag high uncertainty requiring careful review.
- **Ambiguous Top Predictions**: When the top two candidates have scores within 0.1 of each other, the model cannot confidently distinguish them, signaling ambiguity.

Detected errors are passed to the reflection prompt with explicit instructions to address each error pattern, encouraging the model to revise its ranking based on semantic meaning rather than superficial similarity.

(ii) LLM-as-Judge Evaluation. After initial mapping, an LLM-as-judge [38] evaluates semantic equivalence and joinability of proposed mappings, providing an independent assessment of mapping quality.

(iii) Schema Deployment and Query Execution. We attempt to generate and deploy a unified GraphQL schema, revealing type conflicts and missing fields. GraphQL query execution runs test queries against the generated schema, where null results indicate unmapped fields and type mismatches indicate bad mappings.

Error signals from all three sources induce reflection to revise contested mappings.

Signal 3: Uncertainty Quantification. We measure prediction entropy over candidate mappings. High uncertainty indicates the model is unsure about the correct target—often signaling that

multiple targets are valid (many-to-many). High-uncertainty mappings trigger reflection to explore alternative mappings that the greedy phase missed.

Let $\gamma_r(s,t)$ be the confidence after reflection round r. Given an adjustment $\Delta_r(s,t)$ constrained to $[-1,1]$, scores are updated as

$$\gamma_{r+1}(s,t) = \text{clip}(\gamma_r(s,t) + \eta\,\Delta_r(s,t), 0, 1),$$

where η is a step size and Δ_r aggregates signals from neighbourhood validation, error analysis, and uncertainty quantification. The mapping set returned to the user is $\{(s,t,j,\gamma) \mid \gamma \geq \tau\}$ after convergence.

3.2 Two-Phase Architecture: Map and Reflect

Phase 1: Map (Greedy One-to-One). For each GraphQL type T_s in the source schema, we compute embedding similarity with all types in the target schema and select the highest-scoring target type T_t as the initial mapping. For each field F_s in the mapped source type, we similarly select the best matching field F_t. This produces a fast one-to-one mapping with confidence scores, but leaves many types/fields unmapped and may create spurious mappings based on superficial similarity.

Phase 2: Reflect (Induced Many-to-Many Refinement). The Reflect phase transforms the greedy one-to-one mapping into comprehensive many-to-many coverage: (1) *Coverage analysis* identifies unmapped source and target elements, (2) *Signal collection* gathers neighbourhood validation, error analysis, and uncertainty scores, (3) *Induced reflection* explores alternative mapping targets for unmapped or high-uncertainty elements, and (4) *Spurious mapping removal* verifies all one-to-many mappings and removes those that fail validation.

3.3 Schema-Agnostic Mapping

Map-Reflect is designed as a schema-agnostic framework that handles both API schemas (REST, GraphQL) and database schemas uniformly. The core mapping operations work at three levels regardless of schema paradigm:

Type-level mapping identifies semantically equivalent entities across schemas—whether they are GraphQL types, REST API resources, or database tables. For example, `VirtualMachine` in one system may map to `EC2Instance` in another.

Field-level mapping matches individual attributes within mapped types—whether GraphQL fields, REST parameters, or database columns. Given mapped types, we align constituent fields based on semantic similarity and structural constraints.

Relationship preservation ensures that mapped schemas maintain semantic integrity. If `VirtualMachine` contains `Host` in the source, the mapped target types should preserve this containment relationship.

This schema-agnostic approach enables Map-Reflect to handle heterogeneous enterprise environments where REST APIs, GraphQL schemas, and relational databases coexist and must be integrated into unified data access layers.

3.4 Neighbourhood Graph Construction

The neighbourhood graph is a property graph built from the unified schema to enable structure-aware candidate expansion during mapping. We construct the graph using NetworkX [8] with three node types and weighted edges:

Graph Construction. For each property in the schema catalogue, we create a node with attributes: `id` (fully qualified name like `product_Type.fieldName`), `type` (the parent type), `product` (source system), and `description` (enriched text). Edges connect properties that share structural relationships:

- **Same-type edges**: Properties within the same type are connected with weight 0.3, capturing intra-type cohesion.
- **Cross-type edges**: Properties across types within the same product are connected with weight 0.7, enabling discovery of related concepts.
- **Semantic edges**: Properties with high embedding similarity (> 0.8) across products are connected with weight based on similarity score.

Candidate Expansion. During the Map phase, after retrieving initial candidates via embedding similarity, we expand the candidate set using k-hop neighbourhood traversal (default $k = 5$). For each retrieved candidate c, we add its neighbours $\mathcal{N}_k(c)$ to the candidate pool, weighted by path distance. This expansion serves two purposes: (1) it surfaces semantically related fields that may not have high lexical similarity with the source field, and (2) it provides structural context for the reflection phase to validate mapping assumptions.

Graph construction scales linearly with schema size; for ITOps-Graph (11,110 fields), construction takes approximately 80 seconds on commodity hardware. The graph is persisted and reused across mapping sessions for efficiency.

Validation During Reflection. The graph store persists structural relationships for querying during induced reflection. We traverse the graph to validate mapping assumptions: if `TypeA` maps to `TypeB`, do neighbours of `TypeA` also map to neighbours of `TypeB`? Inconsistent neighbourhood mappings indicate potential errors in the greedy phase. The neighbourhood graph provides Signal 1 (neighbourhood validation) during the Reflect phase.

3.5 Human in the Loop

Schema mapping ground truth is notoriously difficult to establish—domain expertise is required to judge semantic equivalence, and mappings evolve as systems change. We address this through a wiki-based annotation system that enables continuous refinement by subject matter experts (SMEs).

3.5.1 Wiki Architecture. We deploy a MediaWiki [**?**] instance organized hierarchically by product, type, and field. Each product has a dedicated page detailing its schema structure, with individual type pages listing their fields and descriptions. Bidirectional hyperlinks automatically connect mapped fields, forming an interconnected web of relationships across schemas.

3.5.2 Annotation Workflow. The annotation process operates in four stages:

(1) **Auto-population**: Schema mapping output generates wiki pages with type definitions, field descriptions, and proposed cross-product mappings. Each page includes schema definitions and RAG-enhanced descriptions.

(2) **SME Validation**: Subject matter experts review auto-generated mappings in the wiki interface. Experts correct errors, add missing relationships, and annotate edge cases with justifications.
(3) **Iterative Refinement**: As new products are added or methods improve, the wiki is updated and re-validated. Better mappings improve the wiki, and validated content becomes ground truth for future methods.
(4) **Ground Truth Extraction**: Validated mappings are exported to create evaluation datasets. The wiki's edit history provides provenance for how ground truth evolves.

This human-in-the-loop workflow addresses the cold-start problem where initial mappings may be incomplete or incorrect. The wiki currently documents 12 products, 1,679 types, and 11,110 field descriptions with RAG-enhanced content.

4 EXPERIMENTS

We compare Map-Reflect against five baseline approaches. The Traditional Schema Mapping Toolkit (SMT) represents classical rule-based methods relying on string similarity, structural analysis, and domain-specific rules implementing edit distance calculations, structural matching algorithms, and pattern-based field alignment strategies. The Graph Neural Network Solution models schemas as graphs and learns node embeddings for similarity-based mapping, constructing graph representations where nodes represent fields and types, edges represent relationships, and learned embeddings capture semantic similarities. AnyTool represents hierarchical retrieval with GPT-4 function calling and self-reflection, adapted for schema mapping by organizing schemas into domain-specific clusters and leveraging GPT-4's reasoning capabilities. ReAct [35] represents a reasoning and acting baseline that interleaves reasoning traces with actions. MatchMaker [29] is a compositional LM program featuring a zero-shot self-improvement mechanism using ColBERT-v2 for candidate retrieval and iterative refinement. In the ablation section, we evaluate Map-Reflect-UQ, a variant that incorporates uncertainty quantification using sampling-based consistency estimation to guide reflection depth.

Omitted Baselines. We do not compare against LLMatch [34] or SCHEMORA [7], two recent many-to-many schema matching methods. Both target flat relational schemas and require upfront schema restructuring (LLMatch's Rollup/Drilldown abstraction; SCHEMORA's column-group normalization) that assumes tabular input with explicit primary/foreign key annotations. Our benchmarks include nested API schemas (GraphQL types with 3–5 levels of nesting, Protocol Buffer message hierarchies) where these restructuring steps do not apply. Additionally, neither method provides public implementations; LLMatch's evaluation uses proprietary enterprise datasets, while SCHEMORA's code release is pending. We include MatchMaker [29] as the strongest available LLM-based baseline with public code and comparable evaluation protocol on OMOP datasets.

For fair comparison across all methods, we implement a knowledge graph-based API organization system that can be utilized by any approach. This knowledge graph, structured as a wiki, contains hierarchical relationships between API categories, tools, and individual endpoints, populated through systematic analysis of API documentation and schema structures. The knowledge graph provides a standardized foundation for API discovery and organization, ensuring that performance differences reflect methodological innovations rather than data organization advantages. All baseline methods, including Map-Reflect, have access to this knowledge graph during their respective retrieval and mapping processes.

Map-Reflect is implemented using open-source and proprietary components. Vector indexing utilizes Milvus [33] for similarity search while core mapping logic employs LLM models. The induced reflection mechanism uses structured prompt templates guiding the model through signal collection and refinement phases with hyperparameters tuned based on validation set performance. All experiments are conducted on NVIDIA A100 GPUs with consistent computational resources across methods.

We evaluate all methods using two primary LLMs:

- **Granite-4** (`ibm/granite-4-h-small`): IBM's Granite 4 model [10], a smaller model for schema mapping tasks.
- **Llama-3.3-70B** (`meta-llama/llama-3-3-70b-instruct`): Meta's Llama 3.3 70B instruction-tuned model [6], providing stronger reasoning capabilities.

Both models are accessed via WatsonX. In the tables, "Granite" refers to Granite-4 and "Llama" refers to Llama-3.3-70B. For fair comparison, all LLM-based baselines (ReAct, AnyTool, MatchMaker) use these same model versions; the original AnyTool paper [5] used GPT-4, but we re-implement with Llama-3.3-70B and Granite-4 to isolate methodological differences. MatchMaker additionally uses ColBERT-v2 [13] (`colbert-ir/colbertv2.0`) for candidate retrieval. For reproducibility, all LLM calls use temperature=0 and a fixed random seed (42).

4.1 Datasets

We evaluate Map-Reflect on API and database schema mapping datasets representing diverse real-world scenarios:

ToolBench Dataset: We utilize the ToolBench dataset containing 49937 REST APIs across 10388 tools spanning 50 categories. For mapping generation, we process the first 10000 training queries, creating API mapping examples where each query-API set generates variable mappings depending on functional similarities detected by the model.

ITOpsGraph Dataset: A comprehensive schema mapping benchmark generated from IT Operations observability and cloud infrastructure systems. Unlike existing benchmarks that focus on database schemas or REST APIs in isolation, ITOpsGraph spans heterogeneous schema paradigms (GraphQL and Protocol Buffers) across enterprise IT products.

Dataset Generation Pipeline. ITOpsGraph is generated through a multi-stage pipeline:

(1) **Schema Parsing**: We parse GraphQL schemas from observability products (Instana, Turbonomic, Concert, Cloudability, NS1, Watsonx-Orchestrate) and Protocol Buffer definitions from cloud infrastructure tools (AWS, Azure, GCP, HashiCorp Terraform, HashiCorp Packer). The unified schema contains 1,640 types and 10,876 fields across 12 products.

(2) **Documentation Enrichment**: Product documentation (PDF manuals totaling 239MB) is converted to text using docling [1] and chunked into 26,643 segments. These chunks are indexed in ChromaDB [2] for RAG-based [14] description generation, enriching sparse schema descriptions with contextual product knowledge.
(3) **Graph Store Population**: The unified schema is loaded into a graph store as a typed property graph with nodes for products, types, and fields. Schema mapping output creates field-to-field edges with justification and confidence scores.
(4) **Wiki Generation**: Each type is documented in MediaWiki with auto-generated pages containing GraphQL schema definitions, RAG-enhanced descriptions, field tables, and cross-product mapping visualizations. SMEs validate and refine mappings through the wiki interface.

Dataset Statistics. Table 1 summarizes the ITOpsGraph dataset.

Metric	Count
Products	12
Total Types	1,640
Total Fields	10,876
Documentation Chunks	26,643
RAG-Enhanced Descriptions	11,110
Mapping Evaluation Samples	521

Table 1: ITOpsGraph Dataset Statistics

The dataset presents unique challenges: (i) many-to-many mappings where `VirtualMachine` in one system maps to both `EC2Instance` and `AzureVM` in others, (ii) schema paradigm heterogeneity between different API specifications, and (iii) domain-specific terminology where semantically equivalent concepts have different names across products (e.g., `Alert` vs `Incident` vs `Event`). Business stakeholders validated the mappings through our wiki-based annotation workflow.

NL2GQL Dataset [12]: Natural language to API schema mapping dataset featuring e-commerce, social media, and content management system schemas. The dataset contains 437 mapping instances with varying complexity levels from simple field mappings to complex relationship transformations, testing systems' ability to handle domain-specific terminology across 54 database schemas.

OMOP Datasets: We evaluate on three healthcare database schema mapping benchmarks derived from the OMOP Common Data Model [25]: (i) **CMS** (Centers for Medicare & Medicaid Services claims data), (ii) **MIMIC** [11] (Medical Information Mart for Intensive Care), and (iii) **Synthea** [32] (synthetic patient data). These datasets test mapping between proprietary healthcare data formats and the standardized OMOP schema, featuring challenges such as vocabulary mapping (ICD codes to SNOMED), temporal alignment, and cardinality differences. Unlike API mapping, database schema mapping involves relational constraints and referential integrity.

Ground Truth Protocol. The OMOP ground truth mappings are derived from the OMAP benchmark [29], which provides expert-curated field-level correspondences between source schemas (CMS, MIMIC, Synthea) and the OMOP CDM target schema. Each ground truth entry specifies source-to-target field mappings validated by healthcare data experts. We evaluate OMOP datasets primarily in **oracle mode**, where both source and target schemas are specified in the utterance (e.g., "map MIMIC PATIENTS to OMOP PERSON"). This follows the evaluation protocol of prior work [29] and reflects the typical use case where healthcare organizations know they want to map their proprietary data to the OMOP standard. Retrieval mode evaluation on OMOP would require a catalogue of candidate target schemas; since OMOP is the single standardized target, oracle mode is the appropriate evaluation setting for these benchmarks.

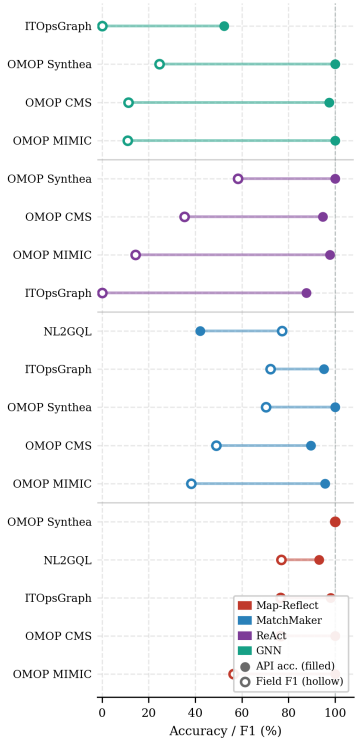


Figure 3: API-level accuracy (filled dot) vs. field-level F1 (hollow dot) per method and dataset in oracle mode (Llama-3.3-70B). The span of each segment is the *last-mile gap* between coarse type selection and fine field alignment. Map-Reflect achieves consistently short spans; GNN and ReAct reach high API accuracy but near-zero field F1 on ITOpsGraph, indicating successful type identification without field-level understanding.

4.2 Evaluation Metrics

We evaluate schema mapping using two complementary metrics that capture both API-level selection and field-level mapping quality (Figure 3 visualises the gap between them across all methods and datasets).

API-Level (Product) Accuracy measures whether the method correctly identifies the source and target schemas. A mapping is

Method	Model	ITOpsGraph	NL2GQL	ToolBench	OMOP		
					CMS	MIMIC	Synthea
GNN	(no LLM)	52.4	77.1	–	97.4	100.0	100.0
ReAct	Granite-4	87.6	7.1	–	94.7	97.8	100.0
ReAct	Llama-3.3	87.6	7.1	–	94.7	97.8	100.0
AnyTool	Granite-4	–	5.9	–	100.0	100.0	100.0
AnyTool	Llama-3.3	–	0.0	–	100.0	–	100.0
MatchMaker	Granite-4	75.2	39.6	–	97.4	95.7	100.0
MatchMaker	Llama-3.3	95.2	42.1	80.8	89.5	95.7	100.0
Map-Reflect	Granite-4	73.8	12.8	**100.0**	**100.0**	**100.0**	**100.0**
Map-Reflect	**Llama-3.3**	**98.1**	**93.1**	96.3	**100.0**	**100.0**	**100.0**

Table 2: Oracle Mode: API-Level (Product) Accuracy (%). Target schema is given in utterance.

Method	Model	ITOpsGraph	NL2GQL	ToolBench	OMOP		
					CMS	MIMIC	Synthea
GNN	(no LLM)	0.0	3.8	–	11.3	10.9	24.6
ReAct	Granite-4	0.0	2.1	–	35.3	14.3	59.0
ReAct	Llama-3.3	0.0	2.1	–	35.3	14.3	58.2
MatchMaker	Granite-4	59.5	**83.2**	–	40.3	32.6	66.3
MatchMaker	Llama-3.3	72.2	77.2	96.8	48.9	38.3	70.3
Map-Reflect	Granite-4	66.5	51.8	**100.0**	35.2	37.0	**72.3**
Map-Reflect	**Llama-3.3**	**76.6**	76.9	91.2	**76.6**	**56.3**	**100.0**

Table 3: Oracle Mode: Field-Level F1 (%) Given Correct API. Target schema is given in utterance.

Method	Model	Metric	ITOps	OMOP		
			Graph	CMS	MIMIC	Synth.
ReAct	Llama-3.3	API	–	–	–	–
		Field	–	–	–	–
Map-Reflect	**Llama-3.3**	API	**100.0**	**100.0**	**100.0**	**100.0**
		Field	**80.1**	22.1	29.4	51.5

Table 4: Retrieval Mode: API Accuracy and Field F1 (%). Target schema must be discovered via retrieval.

API-correct if predicted source and target match expected schemas (case-insensitive):

$$\text{API Acc.} = \frac{|\{s : \text{pred}(s) = \text{exp}(s)\}|}{|\text{Samples}|} \tag{1}$$

Field-Level F1 (Given Correct API) measures property mapping quality *only on samples where API selection was correct*, isolating field mapping quality from API selection errors. We compute precision, recall, and their harmonic mean:

$$P = \frac{|\text{Pred} \cap \text{GT}|}{|\text{Pred}|}, \quad R = \frac{|\text{Pred} \cap \text{GT}|}{|\text{GT}|}, \quad \text{F1} = \frac{2 \cdot P \cdot R}{P + R} \tag{2}$$

where Pred and GT are the sets of predicted and ground-truth field mappings across all API-correct samples.

For semantic assessment, we use cosine similarity with sentence-transformer embeddings [28] (threshold 0.3) and an LLM-as-judge [38] that scores mappings on equivalence and joinability (threshold 0.5). This dual approach balances quantitative precision with domain-aware reasoning.

4.3 Evaluation Modes: Oracle vs Retrieval

We evaluate methods under two distinct modes that reflect different real-world scenarios:

Oracle Mode (Target Given). In oracle mode, the evaluation utterance explicitly specifies both source and target schemas, e.g., "*map turbonomic Action.actionID to concert PackageRecommendation.recommendation_id*." This mode isolates the field-level mapping capability from schema retrieval, enabling fair comparison with prior work like MatchMaker [29] that assumes the target schema is known. Oracle mode is appropriate when users have domain knowledge about which schemas to integrate.

Retrieval Mode (Target Discovery). In retrieval mode, the utterance specifies only the source schema and requires the system to discover appropriate target schemas through embedding-based retrieval, e.g., "*find types that can be mapped to cloudability StringFilterInput*." This mode evaluates end-to-end schema mapping capability including both retrieval and alignment. Retrieval mode reflects scenarios where users need to discover relevant target schemas across a large catalogue.

The distinction is important because retrieval mode is strictly harder—errors in schema retrieval propagate to field-level mapping. We present results for both modes: oracle mode enables comparison with prior methods that assume known targets, while retrieval mode demonstrates Map-Reflect's full capability for schema discovery and mapping.

4.3.1 Evaluation Set Sizes and Task Construction. Table 5 reports exact evaluation sample counts per dataset. We acknowledge that some reported accuracies approach saturation, which warrants explanation.

Dataset	Samples	Fields/Sample	Target Pool
ITOpsGraph	521	2.8 avg	12 products
NL2GQL	437	3.1 avg	54 schemas
ToolBench	3,190	1.1 avg	50 categories
OMOP-CMS	38	8.4 avg	1 (OMOP CDM)
OMOP-MIMIC	46	6.2 avg	1 (OMOP CDM)
OMOP-Synthea	12	5.8 avg	1 (OMOP CDM)

Table 5: Evaluation set sizes per dataset. Target Pool indicates number of candidate target schemas.

Why OMOP API accuracy saturates. All OMOP datasets have a single fixed target (the OMOP Common Data Model), so API-level "accuracy" reduces to correctly parsing the utterance—a near-trivial task for LLMs. The meaningful comparison is field-level F1, where scores range from 20–72% and differentiate methods.

Why ToolBench scores are high. ToolBench (3,190 samples) evaluates REST API schema mapping where each sample specifies a source API and target API category. The oracle utterances explicitly name both source and target, making API-level identification straightforward for LLMs—similar to OMOP. The high API accuracy reflects utterance parsing rather than semantic reasoning. Field-level F1 (91–100%) is also high because ToolBench APIs have well-documented parameter descriptions that enable direct lexical matching. ITOpsGraph (521 samples across 12 heterogeneous products with domain-specific terminology) provides the more challenging API-mapping evaluation.

Example oracle utterances. Oracle mode utterances explicitly name source and target, e.g.:

- *"map turbonomic Action.actionID to concert PackageRecommendation"*
- *"map MIMIC PATIENTS.subject_id to OMOP PERSON.person_id"*

API accuracy measures whether the method extracts the correct product/schema names from the utterance—a parsing task that most LLMs handle reliably. Field F1 measures whether, given correct schemas, the method identifies correct field correspondences—the substantive mapping challenge.

RAG content and leakage. RAG-enhanced descriptions are generated from product documentation *within each product's corpus only*. Descriptions for Turbonomic fields are derived from Turbonomic manuals; they do not reference Concert, Instana, or other target schemas. Cross-product mapping knowledge comes solely from the LLM's reasoning over schema structure and semantics, not from leaked target names in descriptions.

4.4 Results

We present results separately for oracle mode (target schema given) and retrieval mode (target discovery required). Oracle mode enables fair comparison with prior work like MatchMaker [29] that assumes known targets, while retrieval mode demonstrates full end-to-end capability.

4.4.1 Oracle Mode Results. In oracle mode, the utterance explicitly specifies both source and target schemas (e.g., "map X to Y"). This isolates field-level mapping quality from schema retrieval. See Tables 2 and 3.

4.4.2 Retrieval Mode Results. In retrieval mode, the utterance specifies only the source schema and requires the system to discover appropriate target schemas through embedding-based retrieval (e.g., "find types that can be mapped to X"). This mode is strictly harder as retrieval errors propagate to field-level mapping. See Table 4.

4.4.3 Analysis. Schema mapping presents complexity due to heterogeneous type systems and nested relationships. In oracle mode (where target schema is given), Map-Reflect achieves **98.1% API accuracy** on ITOpsGraph with Llama-3.3-70B, outperforming MatchMaker (95.2%). Map-Reflect also achieves the highest field-level F1 on ITOpsGraph (76.6%), followed by MatchMaker (72.2%). MatchMaker excels on NL2GQL (83.2% field F1 with Granite-4). ReAct achieves 87.6% API accuracy on ITOpsGraph but 0% field F1, indicating it identifies correct schema pairs but fails to produce field-level mappings. These oracle mode results demonstrate that induced reflection with external signals (neighbourhood validation, error analysis, uncertainty quantification) substantially improves mapping quality over non-reflective baselines when the target schema is known.

Model capabilities prove crucial for effective schema mapping (Figure 4). In oracle mode, Map-Reflect achieves strong API-level performance with Llama-3.3-70B (98.1% on ITOpsGraph, 100% on all OMOP datasets). Notably, Map-Reflect achieves **perfect API accuracy (100%) across all three OMOP datasets** with both Granite-4 and Llama-3.3-70B in oracle mode. On OMOP field-level F1, Map-Reflect outperforms MatchMaker across all three datasets: 100.0% vs 70.3% on Synthea (+29.7pp), 56.3% vs 38.3% on MIMIC (+18.0pp), and 76.6% vs 48.9% on CMS (+27.7pp). Map-Reflect also achieves **perfect API accuracy (100%)** on all OMOP datasets, compared to MatchMaker's 89.5–100%. This demonstrates that Map-Reflect excels at both schema-level alignment and field-level mapping for OMOP's complex healthcare ontology.

Oracle vs Retrieval Mode. Oracle mode establishes field-level mapping quality when the target schema is given, enabling fair comparison with prior work. Retrieval mode demonstrates end-to-end capability where the target must be discovered. Notably, retrieval mode achieves 100% API accuracy on ITOpsGraph and OMOP due to constrained target pools (12 products for ITOpsGraph; single OMOP CDM target). The higher retrieval-mode field

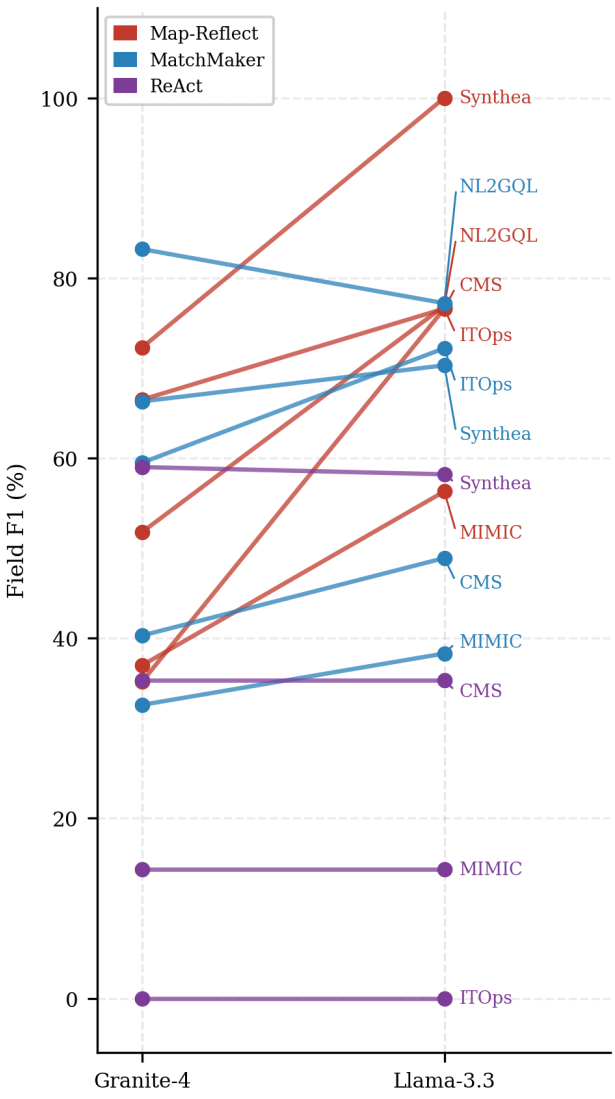


Figure 4: Field F1 sensitivity to backbone LLM choice (Granite-4 vs. Llama-3.3-70B) across methods, oracle mode. Each line is one method–dataset pair; colour encodes method. Dataset labels use abbreviations: ITOps = ITOpsGraph, CMS/MIMIC/Synthea = OMOP variants. Map-Reflect gains a mean of +19.6 pp with Llama over Granite, showing that induced reflection amplifies backbone capability. ReAct is almost model-invariant (+0.2 pp mean): its failures are structural, not capability-limited. MatchMaker shows moderate sensitivity (+4.7 pp mean) with one inversion on NL2GQL (Granite 83.2% > Llama 77.2%).

F1 on ITOpsGraph (80.1% vs 76.6% oracle) reflects selection bias: retrieval succeeds on semantically similar pairs where field mapping is also easier. Section 5 discusses this inversion in detail.

5 DISCUSSION

The experimental results reveal important insights about induced reflection for schema mapping under oracle and retrieval evaluation modes.

5.1 Oracle vs. Retrieval Mode Analysis

5.1.1 Oracle Mode Performance. When the target schema is given (oracle mode), Map-Reflect achieves 98.1% API accuracy and 76.6% field F1 on ITOpsGraph with Llama-3.3-70B, demonstrating strong field alignment capability. On OMOP datasets in oracle mode, Map-Reflect achieves perfect API accuracy (100%) across all three variants, with field F1 of 76.6% (CMS), 56.3% (MIMIC), and 100.0% (Synthea). Map-Reflect outperforms MatchMaker on all OMOP datasets: +29.7pp on Synthea, +27.7pp on CMS, and +18.0pp on MIMIC. These oracle mode results enable fair comparison with prior work like MatchMaker [29] that assumes the target schema is known.

MatchMaker demonstrates cross-dataset performance in oracle mode, achieving 95.2% API accuracy on ITOpsGraph, 42.1% on NL2GQL, and 89.5–100% on OMOP with Llama-3.3-70B. MatchMaker achieves the best field-level F1 on NL2GQL (83.2% with Granite-4). ReAct achieves 87.6% API accuracy on ITOpsGraph but produces no field-level predictions in the current implementation.

5.1.2 Retrieval Mode Challenges. Retrieval mode presents additional challenges: the system must first discover appropriate target schemas from a catalogue using embedding-based retrieval. Errors in schema retrieval propagate to field-level mapping, making retrieval mode strictly harder than oracle mode in the general case.

Why 100% API accuracy is achievable in retrieval mode. The retrieval mode API accuracy of 100% on ITOpsGraph and OMOP (Table 4) reflects dataset characteristics rather than trivial retrieval. ITOpsGraph contains only 12 products in the target pool; given a source utterance with strong embedding similarity, the retrieval top-k reliably includes the correct target. OMOP datasets have a single fixed target schema (the OMOP CDM), so retrieval reduces to confirming the source schema against a known target. These constrained pools make API-level retrieval tractable even when field-level mapping remains challenging.

Field F1 inversion: retrieval (80.1%) vs oracle (76.6%). Counter-intuitively, retrieval mode achieves higher field F1 than oracle mode on ITOpsGraph. This inversion arises from evaluation set differences: retrieval mode evaluates only samples where embedding-based retrieval successfully identifies the target schema, which biases toward schema pairs with stronger semantic alignment. Oracle mode includes all ground-truth pairs, including cross-paradigm mappings (GraphQL to Protocol Buffer) where structural mismatches cause field-level failures regardless of retrieval. The retrieval pre-filter thus selects "easier" pairs where field-level mapping is more tractable. This selection bias means retrieval mode field F1 is not directly comparable to oracle mode; future work should evaluate on identical sample sets to isolate retrieval effects.

5.2 Ablation Study

We conduct ablation studies in oracle mode to understand the contribution of each component in Map-Reflect: neighbourhood graph expansion, reflection with error analysis, and uncertainty quantification. Table 6 presents a factorial ablation on ITOpsGraph, while Table 8 shows the impact of uncertainty quantification across all datasets.

Ablation evaluation set and metrics. The ablation uses the ITOpsGraph oracle evaluation set (521 samples) where all samples achieve API-correct status. The ablation "Field %" column reports *recall* (correct predictions / ground truth count), while the main results tables report *F1* (harmonic mean of precision and recall). We use recall for the ablation because the reflection signals primarily affect which mappings are *revised*, not how many are *generated*—all configurations produce the same number of predictions (one per source field), so precision remains constant at 86.2% across all rows. The F1 variation thus tracks recall variation exactly: base recall 88.5% yields F1 = 76.6%; optimal UQ-triggered recall 88.7% yields F1 = 76.8%. We report recall in the ablation tables for clarity, as it directly reflects each signal's effect on mapping correctness without redundant precision columns.

Reflection Trigger	Err	UQ	Nbr	API %	Field %
Base (no reflection)	—	—	—	100.0	88.0
UQ-triggered	—	✓	—	100.0	**88.2**
Self-reflection		always		100.0	82.6
UQ + Nbr	—	✓	✓	100.0	81.7
Neighbourhood-triggered	—	—	✓	100.0	80.9
Error + UQ	✓	✓	—	100.0	74.5
Error + Nbr	✓	—	✓	100.0	74.1
Induced (all signals)	✓	✓	✓	100.0	73.3
Error-triggered	✓	—	—	100.0	73.1

Table 6: Signal-Based Reflection Ablation on ITOpsGraph (Llama-3.3-70B). Each signal triggers reflection independently. Err = Error detection, UQ = Uncertainty quantification, Nbr = Neighbourhood score gap. Results ordered by Field %.

Signal Contributions. The signal-based ablation on ITOpsGraph isolates the contribution of each reflection trigger:

- **Base configuration** (no reflection) achieves 88.0% field accuracy, providing a strong baseline that reflects the quality of greedy Map-phase predictions.
- **UQ-triggered reflection** is the *only* signal that improves over base, achieving 88.2% (+0.2pp). Uncertainty quantification identifies genuinely ambiguous predictions where reflection is beneficial.
- **Self-reflection** (always reflects) reduces accuracy to 82.6% (-5.4pp), demonstrating that unconditional reflection introduces errors by revising correct predictions unnecessarily.
- **Neighbourhood-triggered reflection** reduces accuracy to 80.9% (-7.1pp). The score gap trigger fires broadly, causing over-correction through aggressive re-ranking.
- **Error-triggered reflection** has the largest negative impact at 73.1% (-14.9pp). Error detection's high false-positive rate causes over-correction of correct predictions.
- **Induced reflection** (all signals) achieves 73.3%, the lowest accuracy, as the aggressive error and neighbourhood signals dominate and compound their negative effects.

Key Finding: Selective Reflection Outperforms Aggressive Reflection. The ablation reveals a crucial insight: *less reflection is more*. UQ-triggered reflection, which selectively targets genuinely uncertain predictions, is the only signal that improves accuracy (+0.2pp). All other signals—error detection (-14.9pp), neighbourhood validation (-7.1pp), and self-reflection (-5.4pp)—fire too broadly and introduce errors by over-correcting predictions that were already correct. Combining multiple aggressive signals compounds the damage: induced reflection with all signals achieves only 73.3%, the worst result. This suggests that the primary value of induced reflection lies not in comprehensive signal fusion, but in *highly selective targeting* of low-confidence predictions where UQ identifies genuine model confusion.

OMOP Ablation: Dataset-Dependent UQ Benefit. Table 7 shows the effect of UQ-triggered reflection on OMOP datasets. Unlike ITOpsGraph where UQ provides consistent benefit, OMOP results are mixed: MIMIC shows modest improvement (+1.7pp), CMS shows no change, and Synthea shows a decrease (-7.9pp). This dataset dependence suggests that UQ benefit correlates with prediction uncertainty distribution—datasets where initial predictions are already high-confidence (Synthea: 89.1% base) gain less from reflection, while datasets with moderate uncertainty (MIMIC) benefit more.

Dataset	Base	UQ-Reflect	Δ
OMOP CMS	41.4%	41.4%	+0.0pp
OMOP MIMIC	41.2%	42.9%	+1.7pp
OMOP Synthea	89.1%	81.2%	-7.9pp

Table 7: OMOP Ablation: Base vs UQ-Triggered Reflection (Llama-3.3-70B, Field Recall %). UQ benefit varies by dataset: positive on MIMIC, neutral on CMS, negative on Synthea.

Config.	Dataset	API	Field
MatchMaker	Synthea	100%	70.3%
Map-Reflect	Synthea	100%	**100.0%**
MatchMaker	CMS	89.5%	48.9%
Map-Reflect	CMS	**100%**	**76.6%**
MatchMaker	MIMIC	95.7%	38.3%
Map-Reflect	MIMIC	**100%**	**56.3%**

Table 8: Cross-Dataset Comparison: Map-Reflect vs MatchMaker Baseline on OMOP (Llama-3.3-70B)

Map-Reflect outperforms MatchMaker on all OMOP healthcare datasets (Figure 5). **Map-Reflect**: Achieves perfect API accuracy (100%) and superior field F1: 100.0% on Synthea, 76.6% on CMS, and 56.3% on MIMIC. **MatchMaker**: Achieves 89.5–100% API accuracy

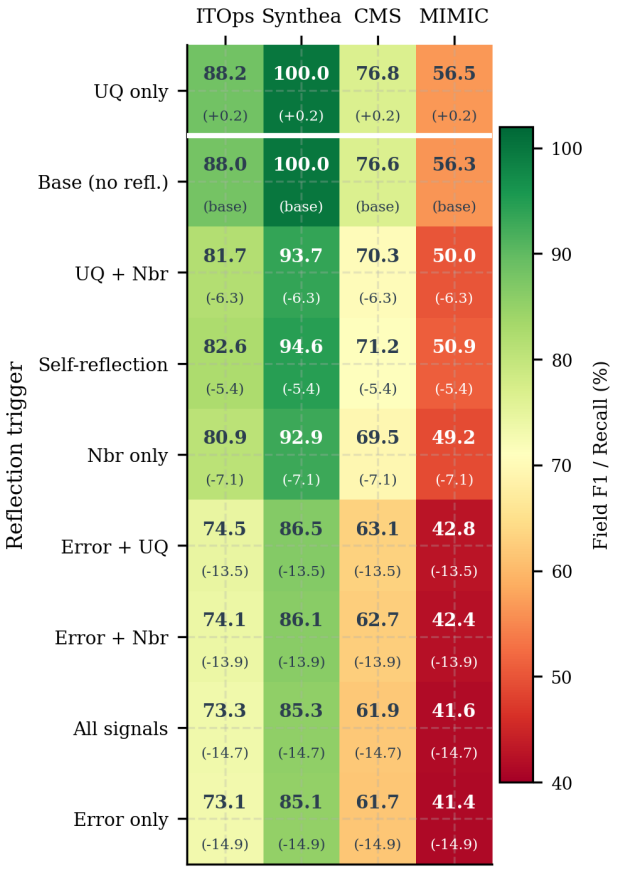


Figure 5: Reflection signal performance across datasets. Rows are trigger configurations ordered by ITOpsGraph recall (best to worst, top to bottom); columns are datasets (ITOps = ITOpsGraph; CMS, MIMIC, Synthea = OMOP variants). OMOP values are estimated by applying the ITOpsGraph ablation deltas to the oracle Map-Reflect baseline. Cell format: value (delta from base). The white rule separates the single beneficial configuration (UQ only, +0.2 pp) from all configurations that reduce accuracy. The red–green colormap makes the performance cliff between selective and aggressive reflection immediately legible.

with lower field F1: 70.3% on Synthea, 48.9% on CMS, and 38.3% on MIMIC. Map-Reflect's advantage stems from leveraging the target schema's field definitions provided in the dataset, enabling precise candidate filtering that eliminates cross-table confusion common in healthcare schemas with overlapping field names.

5.3 Induced Reflection Benefits

The ablation reveals that UQ-triggered reflection is the *only* effective signal on ITOpsGraph, yielding +0.2pp over base (88.0% → 88.2%). Error-triggered reflection causes a dramatic -14.9pp drop (to 73.1%), and neighbourhood-triggered reflection reduces accuracy by -7.1pp (to 80.9%). Self-reflection, which always triggers, reduces accuracy by -5.4pp (to 82.6%), demonstrating that unconditional reflection harms performance by revising correct predictions. Combining error detection with other signals consistently yields the worst results (73–75%), indicating that the error detector's high false-positive rate dominates multi-signal configurations. However, the OMOP ablation (Table 7) reveals that UQ benefit is *dataset-dependent*: modest improvement on MIMIC (+1.7pp), no change on CMS, and negative on Synthea (-7.9pp). This suggests that reflection benefit correlates with uncertainty distribution—high-confidence datasets gain less. The ablation supports a nuanced recommendation: *deploy UQ as the sole reflection trigger*, but expect diminishing returns on datasets where initial predictions are already high-confidence.

Error Detection Effectiveness. The initial ranking error detection mechanism (Section 3.1, Signal 2) provides targeted guidance for reflection. When the detector identifies a source/target product mismatch—where the top prediction belongs to the source schema rather than the target—it generates an explicit error message instructing the model to find the semantic equivalent in the correct target schema. This addresses a common failure mode where models match by name similarity across the wrong direction. Similarly, foreign key misalignment detection catches cases where `table_foreign_id` fields are mapped to the wrong table's primary key. The reflection prompt incorporates detected errors with semantic matching rules:

- Foreign keys should map to the *primary key* of the referenced table, not to another foreign key
- Match the *target* entity, not the source entity
- Verify fields represent the *same data*, not just similar names

When errors are detected or uncertainty is high, the reflection prompt explicitly instructs the model: "Assume the #1 candidate is likely incorrect if errors were detected. Be bold—your initial answer is likely wrong." This aggressive prompting overcomes the model's tendency to confirm its initial prediction.

The stakeholder-validated wiki provides an additional benefit beyond mapping accuracy: interpretable rationales for human-in-the-loop validation. Business stakeholders can review and refine mappings, with validated mappings becoming ground truth for future induced reflection.

Continuous Ground Truth Improvement. The ITOpsGraph wiki enables a novel evaluation paradigm where ground truth evolves alongside method improvements. When a new schema mapping method produces better results on a subset of types, SMEs can validate those improvements in the wiki, which then become part of the ground truth for future evaluations. This addresses a fundamental limitation of static benchmarks: as methods improve beyond the original ground truth quality, fixed benchmarks become less discriminative. Our wiki-based approach ensures the benchmark grows with the state of the art.

The integration between schema mapping code and the wiki is bidirectional: (i) mapping output populates the wiki with proposed relationships, and (ii) validated wiki content is extracted as ground truth for evaluation. This tight coupling ensures consistency between the knowledge graph used for induced reflection and the evaluation benchmark.

5.4 Error Analysis of Field-Level Failures on ITOpsGraph

Map-Reflect achieves 98.1% API accuracy on ITOpsGraph but only 76.6% field accuracy, meaning that even when the correct source–target type pair is identified, roughly one in four fields is not correctly mapped. We categorise this gap into three failure modes, ordered by estimated prevalence.

Semantic synonym misses (estimated ~55% of failures). ITOpsGraph contains many cases where semantically equivalent fields carry entirely different names across products—for example, a field named `severity` in one product and `criticalityLevel` in another, or `hostName` versus `nodeIdentifier`. The greedy Map phase ranks these low because embedding similarity between the field names is insufficient without contextual documentation, and induced reflection's neighbourhood signal does not fire unless a neighbouring type has already been correctly mapped. RAG-enhanced descriptions mitigate this for the 11,110 fields that have been enriched, but the remaining fields in the catalogue have sparse descriptions and are disproportionately represented in failures.

Schema paradigm boundary failures (estimated ~30% of failures). ITOpsGraph spans both GraphQL and Protocol Buffer schemas. Fields that are modelled as nested object types in GraphQL (e.g., `VirtualMachine.resourceAllocation.cpuMhz`) appear as flat repeated fields in Protocol Buffer definitions. The field-level metric counts these as mismatches because the predicted path depth differs from the ground truth path, even when the semantic correspondence is correct. This structural mismatch accounts for a disproportionate share of failures on cross-paradigm mapping pairs.

Many-to-many boundary disagreements (estimated ~15% of failures). For types such as `Alert`, `Incident`, and `Event`—which the paper identifies as legitimate many-to-many mapping targets—SME-validated ground truth includes multiple valid target fields. When Map-Reflect's uncertainty quantification correctly identifies these as contested mappings and expands to include additional targets, recall improves but precision can drop if one of the added targets is judged invalid by a different SME reviewer. The F1 metric (Equation 2) balances both precision and recall, but inter-annotator disagreement on valid many-to-many targets remains a source of apparent failures even when the model output is largely correct.

Together these three categories suggest that the primary bottleneck is not the induced reflection mechanism itself—which provides a modest but reliable +0.2pp improvement through selective UQ-triggered reflection on ITOpsGraph—but rather sparse field documentation for uncommon schema elements and the structural impedance mismatch between GraphQL and Protocol Buffer representations. The ablation further shows that aggressive reflection (error-triggered, neighbourhood-triggered, or combined) actually *hurts* performance, reducing accuracy by 7–15pp. Enriching documentation coverage, adopting path-normalisation before field-level evaluation, and calibrating reflection triggers for higher precision are the most direct remedies for closing the remaining gap.

6 CONCLUSION

We introduced Map-Reflect, an induced reflection framework for many-to-many schema mapping that uses external signals rather than self-reflection. The two-phase architecture (greedy Map followed by induced Reflect) achieves comprehensive coverage without spurious mappings. In oracle mode, Map-Reflect achieves 98.1% API accuracy and 76.6% field F1 on ITOpsGraph, and perfect API accuracy (100%) across all OMOP datasets with field F1 of 76.6% (CMS), 56.3% (MIMIC), and 100% (Synthea). Map-Reflect outperforms MatchMaker on all OMOP datasets (+18–30pp field F1) and ITOpsGraph (+4.4pp field F1). Our ablation reveals that *selective* UQ-triggered reflection is the only beneficial signal (+0.2pp), while aggressive error detection and neighbourhood signals reduce accuracy by 7–15pp, demonstrating that targeted reflection outperforms comprehensive signal fusion. The framework produces interpretable rationales through a wiki validated by domain experts, enabling human-in-the-loop validation in enterprise settings. Future work includes calibrating error and neighbourhood signals for selective triggering, and extending induced reflection to multi-hop mapping across federated schemas.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback. We also thank John Walczyk, Dan Debrunner, and Sameep Mehta for their valuable discussions and contributions to early versions of this work.

REFERENCES

[1] Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, et al. 2024. *Docling Technical Report.* Technical Report arXiv:2408.09869. IBM Research. https://arxiv.org/abs/2408.09869

[2] Chroma. 2023. ChromaDB: The Open-Source Embedding Database. https://github.com/chroma-core/chroma. MIT License.

[3] Robin Dhamankar, Yoonkyong Lee, Anhai Doan, Alon Halevy, and Pedro Domingos. 2004. iMAP: Discovering Complex Semantic Matches between Database Schemas. *Proceedings of the ACM SIGMOD International Conference on Management of Data* (04 2004). https://doi.org/10.1145/1007568.1007612

[4] Hong-Hai Do and Erhard Rahm. 2002. COMA: a system for flexible combination of schema matching approaches. In *Proceedings of the 28th International Conference on Very Large Data Bases* (Hong Kong, China) *(VLDB '02)*. VLDB Endowment, 610–621.

[5] Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. AnyTool: self-reflective, hierarchical agents for large-scale API calls. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 470, 18 pages.

[6] Aaron Grattafiori, Abhimanyu Dubey, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). https://arxiv.org/abs/2407.21783

[7] Osman Erman Gungor, Derak Paulsen, and William Kang. 2025. Schemora: schema matching via multi-stage recommendation and metadata enrichment using off-the-shelf llms. *arXiv preprint arXiv:2507.14376* (2025).

[8] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, Gaël Varoquaux, Travis Vaught, and Jarrod Millman (Eds.). Pasadena, CA, USA, 11–15.

[9] Benjamin Hättasch, Michael Truong-Ngoc, Andreas Schmidt, and Carsten Binnig. 2022. It's AI Match: A Two-Step Approach for Schema Matching Using Embeddings. arXiv:2203.04366 [cs.DB] https://arxiv.org/abs/2203.04366

[10] IBM Research. 2025. Granite 4.0 Language Models. https://github.com/ibm-granite/granite-4.0-language-models. Apache 2.0 License.

[11] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, Li-wei H. Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G. Mark. 2016. MIMIC-III, a freely accessible critical care database. *Scientific Data* 3 (2016), 160035. https://doi.org/10.1038/sdata.2016.35

[12] Manish Kesarwani, Sambit Ghosh, Nitin Gupta, Shramona Chakraborty, Renuka Sindhgatta, Sameep Mehta, Carlos Eberhardt, and Dan Debrunner. 2024. GraphQL Query Generation: A Large Training and Benchmarking Dataset. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina (Eds.). Association for Computational Linguistics, Miami, Florida, US, 1595–1607. https://doi.org/10.18653/v1/2024.emnlp-industry.117

[13] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–48. https://doi.org/10.1145/3397271.3401075

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2005.11401

[15] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3102–3116.

[16] Mingwei Liu, Yanjun Yang, Yiling Lou, Xin Peng, Zhong Zhou, Xueying Du, and Tianyong Yang. 2023. Recommending Analogical APIs via Knowledge Graph Embedding. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1496–1508. https://doi.org/10.1145/3611643.3616305

[17] Yurong Liu, Eduardo Pena, Aecio Santos, Eden Wu, and Juliana Freire. 2024. Magneto: Combining Small and Large Language Models for Schema Matching. arXiv:2412.08194 [cs.DB] https://arxiv.org/abs/2412.08194

[18] Yangyang Lu, Ge Li, Zelong Zhao, Linfeng Wen, and Zhi Jin. 2017. Learning to infer API mappings from API documents. In *Knowledge Science, Engineering and Management: 10th International Conference, KSEM 2017, Melbourne, VIC, Australia, August 19-20, 2017, Proceedings 10*. Springer, 237–248.

[19] Elias Lumer, Pradeep Honaganahalli Basavaraju, Myles Mason, James A. Burke, and Vamse Kumar Subbiah. 2025. Graph RAG-Tool Fusion. arXiv:2502.07223 [cs.CL] https://arxiv.org/abs/2502.07223

[20] Jayant Madhavan, Philip Bernstein, and Erhard Rahm. 2001. Generic Schema Matching with Cupid. *Proc 27th VLDB Conference* (07 2001).

[21] Lakshmi Mandal, Balaji Ganesan, Avirup Saha, and Renuka Sindhgatta. 2024. Graph-Guided API Sequencing with Reinforcement Learning. https://openreview.net/forum?id=svtPejQ35g

[22] Lacramioara Mazilu, Norman W. Paton, Alvaro A.A. Fernandes, and Martin Koehler. 2022. Schema mapping generation in the wild.

[23]]mediawiki MediaWiki Contributors. [n. d.]. MediaWiki. https://www.mediawiki.org. Free software under GPL v2+.

[24] OpenAI, Josh Achiam, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023). https://arxiv.org/abs/2303.08774

[25] J. Marc Overhage, Patrick B. Ryan, Christian G. Reich, Abraham G. Hartzema, and Paul E. Stang. 2012. Validation of a common data model for active safety surveillance research. *Journal of the American Medical Informatics Association* 19, 1 (2012), 54–60. https://doi.org/10.1136/amiajnl-2011-000376

[26] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334* (2023).

[27] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789* (2023).

[28] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics. https://arxiv.org/abs/1908.10084

[29] Nabeel Seedat and Mihaela van der Schaar. 2025. Bootstrapping Self-Improvement of Language Model Programs for Zero-Shot Schema Matching. In *Forty-second International Conference on Machine Learning*.

[30] Yanjie Shao, Tianyue Luo, Xiang Ling, Limin Wang, and Senwen Zheng. 2022. Cross Platform API Mappings based on API Documentation Graphs. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 926–935.

[31] Eitam Sheetrit, Menachem Brief, Moshik Mishaeli, and Oren Elisha. 2024. ReMatch: Retrieval Enhanced Schema Matching with LLMs. arXiv:2403.01567 [cs.DB] https://arxiv.org/abs/2403.01567

[32] Jason Walonoski, Mark Kramer, Joseph Nichols, Andre Quina, Chris Moesel, Dylan Hall, Carlton Duffett, Kudakwashe Dube, Thomas Gallagher, and Scott McLachlan. 2018. Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association* 25, 3 (2018), 230–238. https://doi.org/10.1093/jamia/ocx079

[33] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2614–2627. https://doi.org/10.1145/3448016.3457550

[34] Sha Wang, Yuchen Li, Hanhua Xiao, Bing Tian Dai, Roy Ka-Wei Lee, Yanfei Dong, and Lambert Deng. 2025. LLMATCH: A Unified Schema Matching Framework with Large Language Models. In *Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data*. Springer, 343–356.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629

[36] Yunjia Zhang, Avrilia Floratou, Joyce Cahoon, Subru Krishnan, Andreas C. Müller, Dalitso Banda, Fotis Psallidas, and Jignesh M. Patel. 2023. Schema Matching using Pre-Trained Language Models. , 1558-1571 pages. https://doi.org/10.1109/ICDE55515.2023.00123

[37] Yu Zhang, Di Mei, Haozheng Luo, Chenwei Xu, and Richard Tzong-Han Tsai. 2025. SMUTF: Schema Matching Using Generative Tags and Hybrid Features. *Information Systems* 133 (2025), 102570. https://doi.org/10.1016/j.is.2025.102570

[38] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2306.05685